# FitPick

Digital Wardrobe & Outfit Planner

## Testing, Version Management and System Build Report

Software Engineering · Final Project Submission (Part A)

Author **Nithilan A (nithi0307)**
Repository **github.com/nithi0307/fitpick**
Stack **React 19 + Vite · FastAPI · PostgreSQL 16 · Docker**

# Contents

1. Introduction and scope
2. Testing strategy and test inventory
3. Integration testing
4. Regression testing
5. Mutation testing
6. Version management
7. System building
8. Screenshots of developed functionality
9. Tools and technologies used

# 1. Introduction and scope

**FitPick** is a web application for organising a wardrobe and planning outfits. A user adds clothing items, browses and filters them, combines them into saved outfits, marks favourites and views statistics about what they own. It is built as a **React 19 + Vite** single-page frontend, a **FastAPI** REST backend and a **PostgreSQL 16** database, started together with **Docker Compose**.

This report is the evidence document for the build and quality side of the project. It covers four things:

1. the automated tests — integration, regression and mutation testing — with the terminal output of each run;
2. version management and system building — the Git workflow, repository state and the containerised build;
3. screenshots of every developed screen of the running application;
4. the tools and technologies that were used and why.

Every screenshot in this document is output from this project, captured on the machine the project was developed on. The software design itself (architecture, data flow, ERD, UML) is documented in the companion **FitPick Project Report**.

## 1.1 What was built

| Area | Delivered |
|---|---|
| **Frontend** | 6 screens (Login, Dashboard, My Wardrobe, Add Clothing, Outfit Builder, Profile & Analytics), 7 reusable components, 1 API client module, 1 design-system stylesheet |
| **Backend** | 3 routers (clothing, outfits, stats), 15 REST endpoints, Pydantic schema layer, analytics service, repository layer with two interchangeable implementations |
| **Database** | 4 tables (users, clothing_items, outfits, outfit_items) with schema and seed scripts |
| **Tests** | 46 automated tests across three levels, plus a mutation-testing run |
| **Build** | 3 containers orchestrated by docker-compose.yml, started with a single command |

## 2. Testing strategy and test inventory

Testing concentrates on the backend, because that is where the application's rules live: filtering, validation, storage and the statistics shown on the Dashboard and Analytics screens. The frontend is checked by **ESLint** and a production build on every change, and verified manually against the running API.

### 2.1 Test levels used

| Level | Purpose | Where |
|---|---|---|
| **Integration** | Drive the real FastAPI application over HTTP so routing, validation, serialisation and storage are exercised together. | tests/test_integration_*.py |
| **Regression** | Lock in behaviour for edge cases that were wrong, or easy to get wrong, during development so the same fault cannot return. | tests/test_regression.py |
| **Unit** | Check the analytics calculations directly, without HTTP. These were written in response to the mutation-testing report. | tests/test_analytics_unit.py |
| **Mutation** | Measure how good the tests actually are by deliberately introducing faults and checking that a test fails. | mutmut, configured in setup.cfg |

### 2.2 Test inventory

| Test file | Level | Tests | What it covers |
|---|---|---|---|
| **test_integration_clothing.py** | Integration | 12 | Create, read, update, delete, search, filter, sort, favourite, validation |
| **test_integration_outfits.py** | Integration | 7 | Listing, creating from existing items, rejecting empty or unknown items, delete, favourite |
| **test_integration_stats.py** | Integration | 6 | Dashboard totals, most-worn item, breakdown consistency, wardrobe value |
| **test_regression.py** | Regression | 10 | Ten specific defects and edge cases (Section 4) |
| **test_analytics_unit.py** | Unit | 11 | Counting, ordering, date boundaries, rounding, empty-wardrobe behaviour |
| **Total** | | 46 | |

### 2.3 How the suite is run

*From the backend/ directory*

```
$ python3.13 -m venv .venv
$ .venv/bin/pip install -r requirements-dev.txt
$ .venv/bin/python -m pytest               # all 46 tests
$ .venv/bin/mutmut run                     # mutation testing
```

Tests never touch a database. The repository factory in `repositories/__init__.py` returns the in-memory implementation unless `DATABASE_URL` is set, so the whole suite runs in well under a second and gives the same result on any machine.

*backend/conftest.py — every test starts from a freshly seeded wardrobe*

```
import pytest
from fastapi.testclient import TestClient

import repositories
from main import app


@pytest.fixture
def client() -> TestClient:
    repositories.reset()
    with TestClient(app) as test_client:
        yield test_client
    repositories.reset()
```

## 3. Integration testing

The integration tests use FastAPI's `TestClient`, which sends real HTTP requests into the real application object. Nothing is mocked: a request passes through the router, Pydantic validation, the repository and back out as JSON, exactly as it does for the React frontend. This is what makes them **integration** rather than unit tests — several modules are proved to work together.

### 3.1 What each group covers

- **Clothing (12 tests) —** the full CRUD lifecycle, plus the query parameters the My Wardrobe screen sends: search text, category, season and sort order. Two tests confirm that invalid input is rejected with HTTP 422 rather than being stored.
- **Outfits (7 tests) —** creating an outfit from existing wardrobe items, and the two rules that protect the data: an outfit must contain at least one item, and every item id must exist.
- **Statistics (6 tests) —** the figures on the Dashboard are cross-checked against the wardrobe itself, so the totals cannot drift away from the underlying data.

*backend/tests/test_integration_clothing.py — a representative test*

```
def test_add_item_then_read_it_back(client):
    created = client.post("/clothing", json=NEW_ITEM)
    assert created.status_code == 201
    item_id = created.json()["id"]

    fetched = client.get(f"/clothing/{item_id}")
    assert fetched.status_code == 200
    assert fetched.json()["name"] == "Linen Shorts"
    assert fetched.json()["brand"] == "Arket"
```

The assertion in `test_breakdown_adds_up_to_the_total` is deliberately a relationship rather than a fixed number, so it keeps working as the seeded wardrobe changes:

*backend/tests/test_integration_stats.py*

```
def test_breakdown_adds_up_to_the_total(client):
    stats = client.get("/stats/dashboard").json()
    assert sum(row["count"] for row in stats["breakdown"]) == stats["total_items"]
```

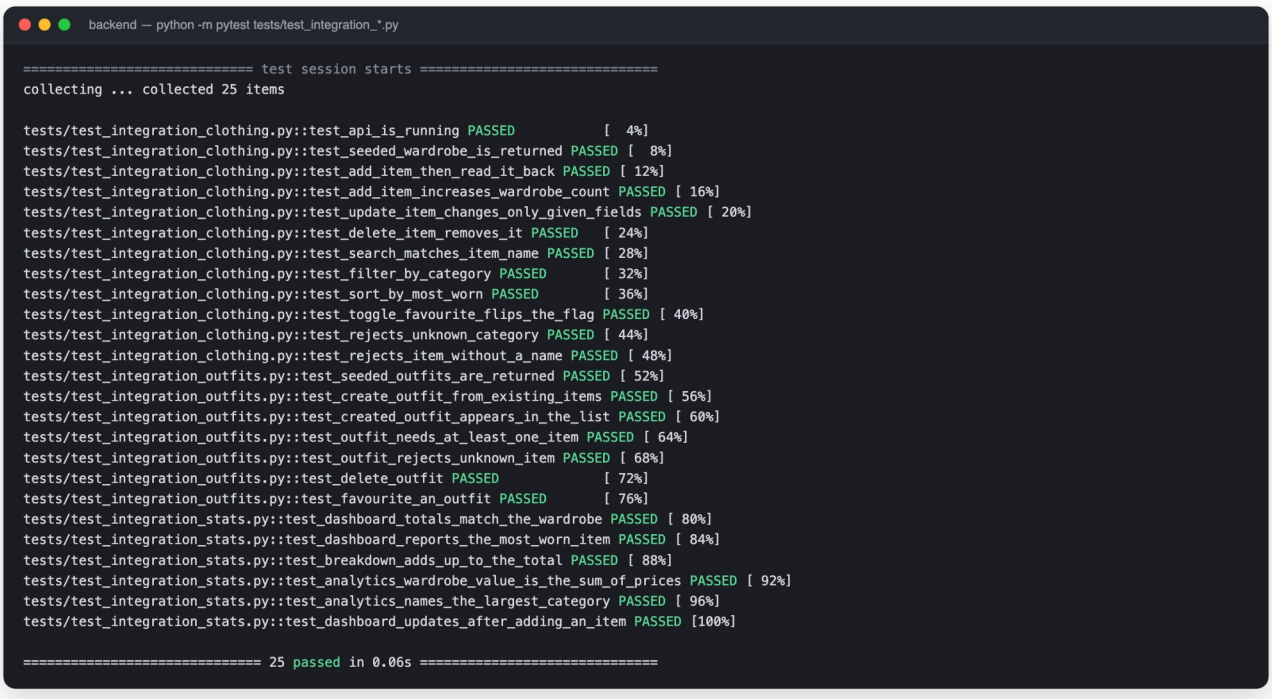



*Figure 1 — The 25 integration tests passing. Each name states the behaviour it proves.*

## 4. Regression testing

A regression test exists to stop a fault coming back. Each of the ten tests below was written after a specific edge case was found during development; they run on every change, so if the behaviour is ever broken again the suite fails immediately.

| ID | Behaviour locked in | Why it was needed |
|---|---|---|
| **REG-01** | Search matches the brand as well as the item name | Search originally looked at the name only, so searching “Levi” returned nothing |
| **REG-02** | Deleting an unknown id returns HTTP 404 | A missing item must not be reported as a successful delete |
| **REG-03** | Category “All” is treated as “no filter” | The literal string was once matched against the category column |
| **REG-04** | Analytics survives an empty wardrobe | Average wears divided by the item count and crashed on a new account |
| **REG-05** | Dashboard survives an empty wardrobe | max() over an empty list raises ValueError |
| **REG-06** | Favouriting twice restores the original state | The toggle must be symmetric |
| **REG-07** | A partial update keeps the fields it was not given | An update once blanked every omitted field |
| **REG-08** | Search is case-insensitive | “NAVY” and “navy” must return the same items |
| **REG-09** | Counts are ordered largest-first | The breakdown chart depends on descending order |
| **REG-10** | Deleting an item updates the statistics | Statistics must never be served from a stale copy |

*backend/tests/test_regression.py — REG-04 and REG-05 both use an empty wardrobe*

```
def test_reg04_analytics_handles_an_empty_wardrobe(empty_wardrobe):
    """Average wears divided by the item count and crashed on a new account."""
    clothing, _ = empty_wardrobe
    stats = analytics_service.analytics_stats(clothing)
    assert stats.average_wears == 0.0
    assert stats.wardrobe_value == 0.0
    assert stats.top_category == "—"


def test_reg05_dashboard_handles_an_empty_wardrobe(empty_wardrobe):
    """max() over an empty wardrobe raised ValueError on the dashboard."""
    clothing, outfits = empty_wardrobe
    stats = analytics_service.dashboard_stats(clothing, outfits)
    assert stats.total_items == 0
    assert stats.most_worn == "—"
    assert stats.breakdown == []
```

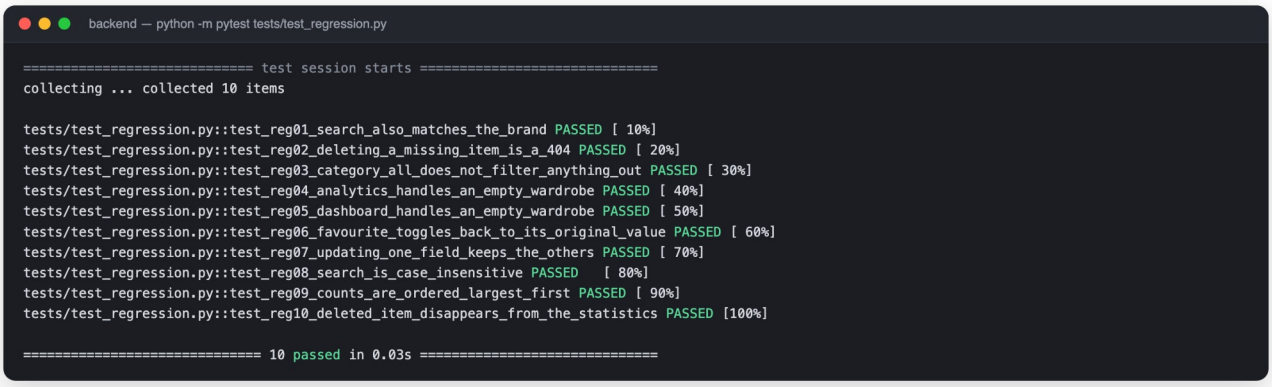



*Figure 2 — All ten regression tests passing.*

# 5. Mutation testing

A suite that passes proves the code runs; it does not prove the tests would notice if the code were wrong. **Mutation testing** answers that question directly. The tool makes small deliberate changes to the source — a >= becomes >, a - becomes + — and runs the suite against each one. If a test fails, the mutant is **killed**. If every test still passes, the mutant **survived**, which means that line of behaviour is not really being checked.

## 5.1 Setup

The tool is `mutmut`. It is pointed at `services/analytics_service.py` — the module that calculates every figure on the Dashboard and Analytics screens. It is the right target because it is pure logic with no I/O, so a wrong number there would be visible to the user but would not crash anything.

*backend/setup.cfg*

```
[mutmut]
paths_to_mutate=services/analytics_service.py
runner=.venv/bin/python -m pytest -x -q --no-header
tests_dir=tests/
```

## 5.2 First run

Running the tool against the integration and regression suites generated 30 mutants:

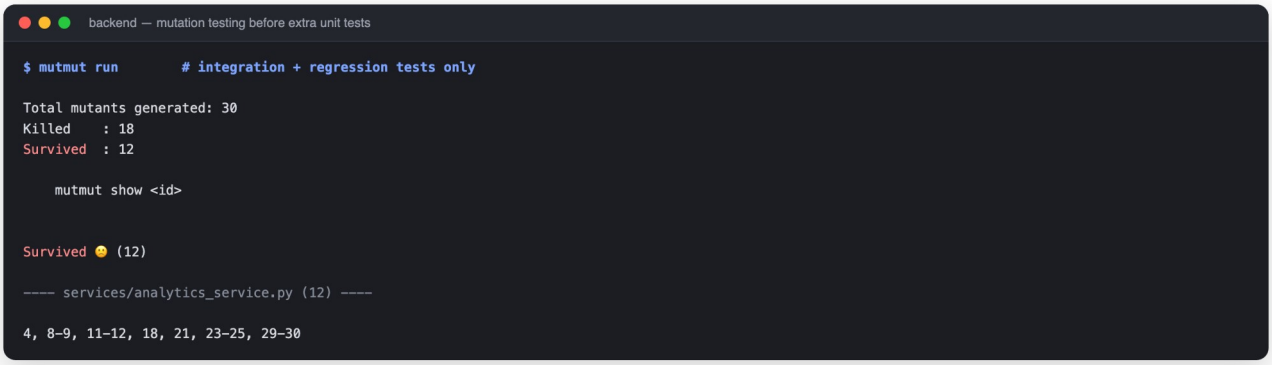



*Figure 3 — First run: 18 of 30 mutants killed, a mutation score of 60%.*

## 5.3 Analysing the twelve survivors

Each surviving mutant was inspected with mutmut show, and each one pointed at a real gap:

| # | Change the tool made | Gap it exposed |
|---|---|---|
| 4 | Tie-break key (-count, label) → (-count, count) | Nothing checked the order of two categories with equal counts |
| 8 | date.today() - timedelta(7) → + | “Added this week” was never asserted against a known number |
| 9 | timedelta(days=7) → days=8 | The seven-day boundary itself was untested |
| 11 | created_at >= cutoff → > | The inclusive edge of the boundary was untested |
| 12 | created_at and … → created_at or … | No item in the fixtures had a missing date |
| 18 | most_worn_count fallback 0 → 1 | The empty-wardrobe test checked the name but not the count |
| 21 | brands = _counts(…) → None | No test read favourite_brand at all |
| 23 | round(value, 2) → round(value, 3) | Prices in the fixtures had no third decimal |
| 24 | total_wears / len(items) → * | The average was never asserted against a known value |
| 25 | round(average, 1) → round(average, 2) | No fixture produced more than one decimal place |
| 29 | brands[0] → brands[1] | No test had two different brands |
| 30 | Placeholder “—” → “XX—XX” | The empty-state placeholder was never asserted |

## 5.4 Closing the gaps

Eleven unit tests were added in `tests/test_analytics_unit.py`, each one written to kill a specific survivor. They are genuine tests, not padding — for example, the date-boundary test pins down exactly which items count as “added this week”:

*backend/tests/test_analytics_unit.py — kills mutants 8, 9 and 11*

```
def test_added_this_week_includes_exactly_the_last_seven_days(wardrobe):
    """The boundary is seven days inclusive: an eight-day-old item is old."""
    clothing, outfits = wardrobe
    add(clothing, "Today", days_old=0)
    add(clothing, "Exactly seven days", days_old=7)
    add(clothing, "Eight days", days_old=8)
    assert analytics_service.dashboard_stats(clothing, outfits).added_this_week == 2
```

*backend/tests/test_analytics_unit.py — kills mutants 24 and 25*

```
def test_average_wears_is_the_mean_rounded_to_one_decimal(wardrobe):
    """1 + 1 + 2 wears over three items is 1.333…, shown as 1.3."""
    clothing, _ = wardrobe
    add(clothing, "A", worn=1)
    add(clothing, "B", worn=1)
    add(clothing, "C", worn=2)
    assert analytics_service.analytics_stats(clothing).average_wears == 1.3


def test_average_wears_divides_rather_than_multiplies(wardrobe):
    clothing, _ = wardrobe
    add(clothing, "A", worn=3)
    add(clothing, "B", worn=5)
    assert analytics_service.analytics_stats(clothing).average_wears == 4.0
```

backend — python -m pytest tests/test_analytics_unit.py

```
============================= test session starts ==============================
collecting ... collected 11 items

tests/test_analytics_unit.py::test_equal_counts_are_ordered_alphabetically PASSED [  9%]
tests/test_analytics_unit.py::test_added_this_week_includes_exactly_the_last_seven_days PASSED [ 18%]
tests/test_analytics_unit.py::test_items_without_a_date_are_not_counted_as_recent PASSED [ 27%]
tests/test_analytics_unit.py::test_empty_wardrobe_reports_a_zero_wear_count PASSED [ 36%]
tests/test_analytics_unit.py::test_average_wears_is_the_mean_rounded_to_one_decimal PASSED [ 45%]
tests/test_analytics_unit.py::test_average_wears_divides_rather_than_multiplies PASSED [ 54%]
tests/test_analytics_unit.py::test_wardrobe_value_is_rounded_to_two_decimals PASSED [ 63%]
tests/test_analytics_unit.py::test_favourite_brand_is_the_most_common_brand PASSED [ 72%]
tests/test_analytics_unit.py::test_items_without_a_brand_are_ignored PASSED [ 81%]
tests/test_analytics_unit.py::test_colour_chart_counts_every_colour PASSED [ 90%]
tests/test_analytics_unit.py::test_empty_wardrobe_shows_a_placeholder_instead_of_a_brand PASSED [100%]

============================== 11 passed in 0.01s ==============================
```

*Figure 4 — The eleven mutation-driven unit tests.*

## 5.5 Second run

backend — mutation testing after extra unit tests

```
$ mutmut run       # full suite, including the mutation-driven unit tests

Total mutants generated: 30
Killed    : 30
Survived  : 0
Timeout   : 0

Mutation score: 100.0%

To apply a mutant on disk:
    mutmut apply <id>

To show a mutant:
    mutmut show <id>
```

*Figure 5 — Second run: all 30 mutants killed, a mutation score of 100%.*

**Outcome**

Mutation score rose from 60% (18/30) to 100% (30/30).
The suite grew from 35 to 46 tests, and the eleven new tests were chosen by evidence rather than guesswork.
No production code had to change: the code was already correct, but twelve of its behaviours were unverified.

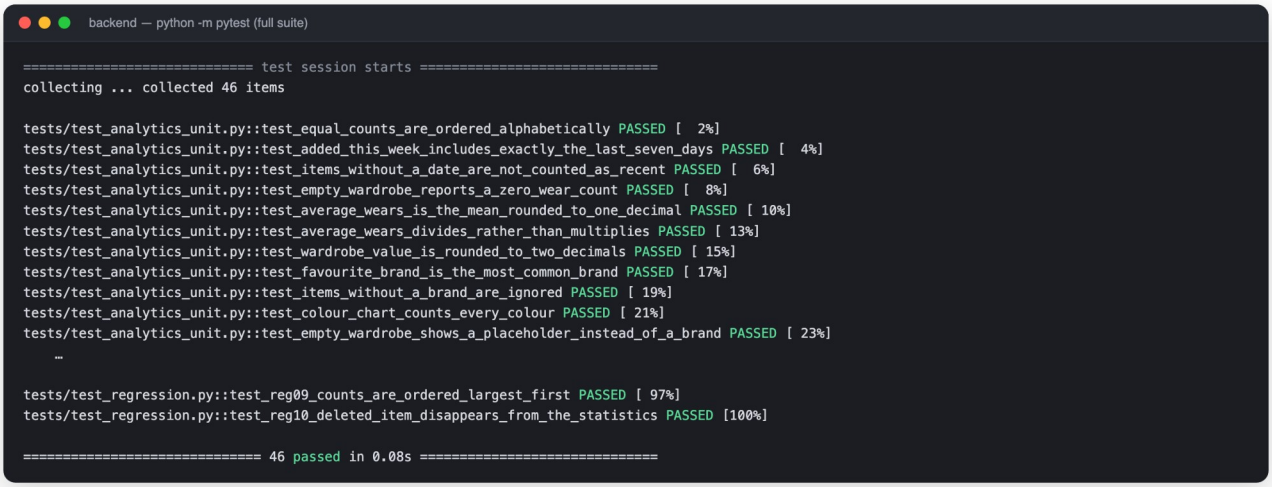



*Figure 6 — The complete suite: 46 tests passing.*

# 6. Version management

The project is version-controlled with **Git** and hosted on **GitHub** at `github.com/nithi0307/fitpick`. Work is tracked as GitHub issues prioritised with MoSCoW on a project board.

## 6.1 Branching strategy

| Branch | Role |
|---|---|
| `main` | Always runnable. Every commit on it builds and passes the test suite. |
| `feature-<name>` | One branch per feature or issue, merged back into main when complete. |

## 6.2 Commit history

Commits are small and each one has a single purpose, so the history reads as a description of how the project was built:

fitpick — git history and branches

```
$ git log --oneline --graph --all --decorate

* 86aec45 (HEAD -> main) Add DFD, ERD, use case, class and sequence diagrams; update the architecture diagram
* 17b8c98 Redesign the frontend: new design system, six rebuilt screens and SVG garment illustrations
* 5caebc1 Expand backend: PostgreSQL repository, analytics service, filtering and stats endpoints
* c0bec1e (origin/main, origin/feature-login, feature-login) Add UI screen captures and the Software Design Document
* 0e48389 Write project README with Software Design section and add design docs index
* bf5128c Add high-level architecture diagram (Draw.io source and PNG export)
* 812e2a6 Structure backend into routers, schemas and repositories; add database schema and seed data
* 6d679e9 Implement app routing, shared layout components and the six main pages
* bf360e0 Add .gitignore and remove accidentally committed dependencies and OS files
* c1c56fe Complete DA1 project setup
* 5e52904 Complete README documentation
* 6004ec5 Initial project setup

$ git branch -a
  feature-login
* main
  remotes/origin/feature-login
  remotes/origin/main

$ git remote -v
origin  https://github.com/nithi0307/fitpick.git (fetch)
origin  https://github.com/nithi0307/fitpick.git (push)
```

*Figure 7 — Commit graph, local and remote branches, and the configured remote.*

## 6.3 Repository hygiene

Early in the project a dependency folder and macOS metadata files were committed by accident. They were removed from tracking and a `.gitignore` now keeps the repository to source files only:

*fitpick/.gitignore*

```
# Dependencies
node_modules/

# Build output
dist/
build/

# Python
__pycache__/
*.pyc
.venv/
venv/

# Environment
.env
.env.*

# OS / editor
.DS_Store
.vscode/
.idea/

# Logs
*.log
.mutmut-cache
mutants/
.pytest_cache/
```

## 6.4 The repository on GitHub

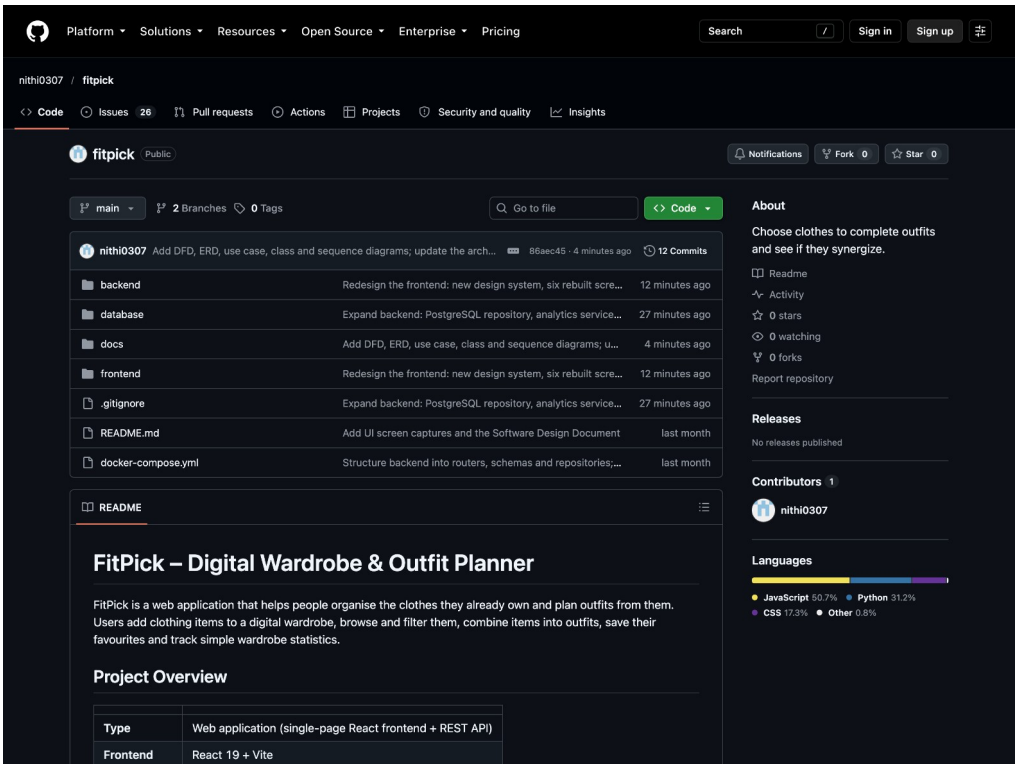



*Figure 8 — The repository: 12 commits, 2 branches, a single contributor, and the README rendered on the landing page.*

## 6.5 Issue tracking

Requirements are tracked as GitHub issues and prioritised with MoSCoW on the FitPick Development Board (15 Must Have, 10 Should Have):

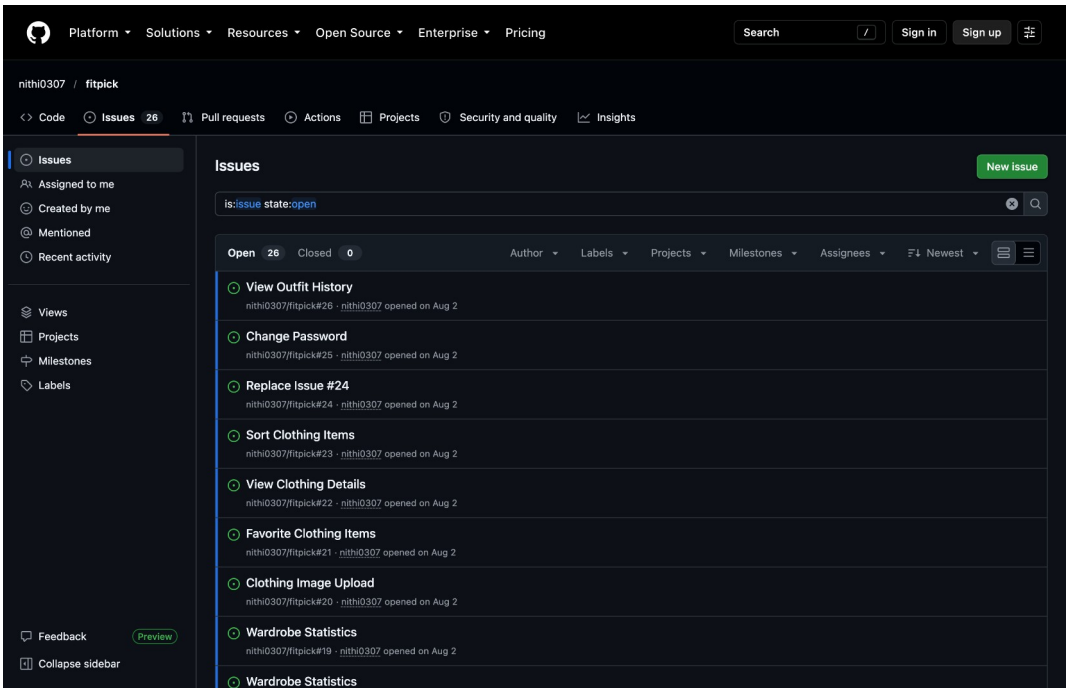



*Figure 9 — The issue list used as the product backlog.*

# 7. System building

The whole system is built and started by one command. `docker-compose.yml` defines three services; each is built from its own Dockerfile and connected on a private network, so a new machine needs nothing installed except Docker.

*fitpick/docker-compose.yml*

```
services:
  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    volumes:
      - ./frontend:/app
      - /app/node_modules
    depends_on:
      - backend

  backend:
    build: ./backend
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app
    environment:
      DATABASE_URL: postgresql://postgres:postgres@database:5432/fitpick
    depends_on:
      - database

  database:
    image: postgres:16
    container_name: fitpick-db
    restart: always
    environment:
      POSTGRES_DB: fitpick
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5433:5432"
    volumes:
      - ./database/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql
      - ./database/seed.sql:/docker-entrypoint-initdb.d/02-seed.sql
      - fitpick-db-data:/var/lib/postgresql/data

volumes:
  fitpick-db-data:
```

## 7.1 Building and starting the stack

The database port is published as 5433 on the host because a local PostgreSQL already occupies 5432; inside the container it stays on the standard 5432, so no application configuration changes.

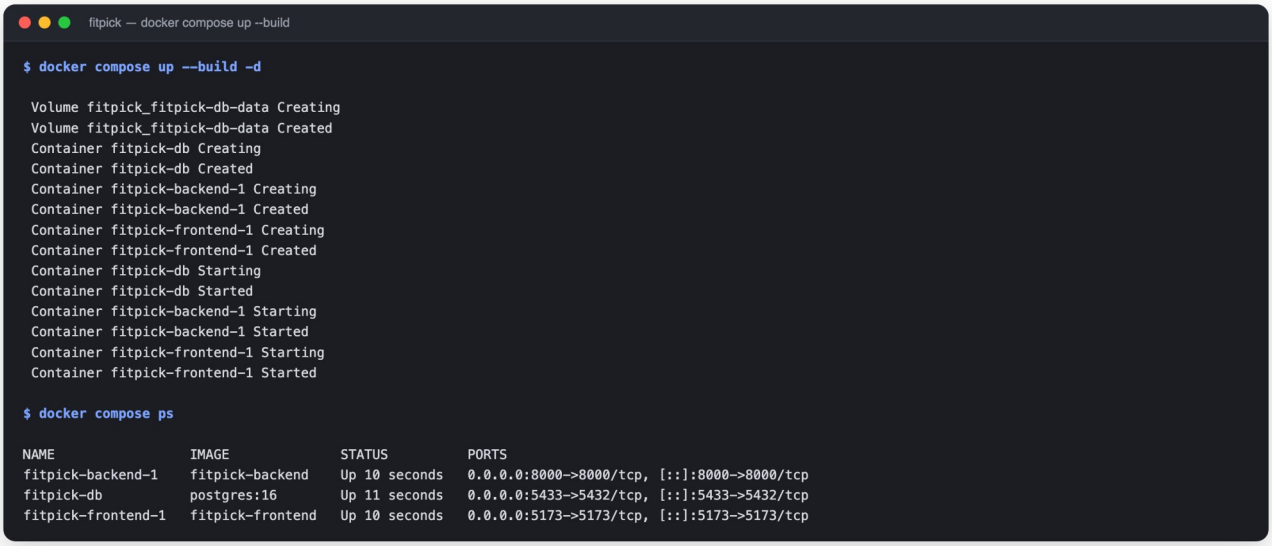



*Figure 10 — docker compose up --build: the three containers built and running.*

## 7.2 The database is created and seeded automatically

`schema.sql` and `seed.sql` are mounted into the PostgreSQL image's initialisation directory, so the first start creates the tables and loads a sample wardrobe with no manual step:

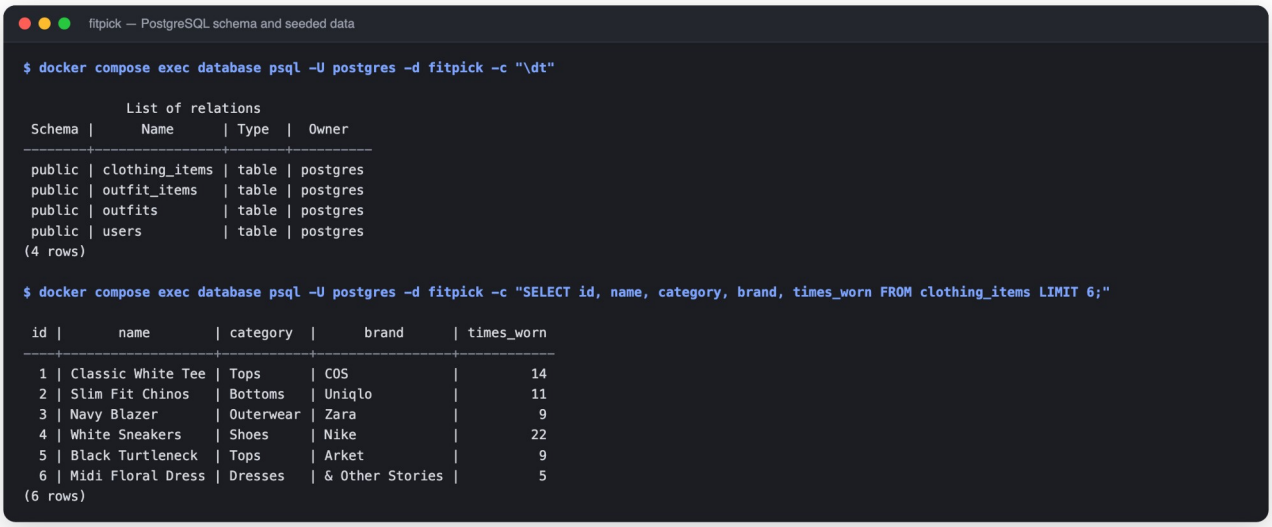



*Figure 11 — The four tables and the seeded clothing items, queried inside the running container.*

## 7.3 The API responds in the built environment

With the stack running, the backend container serves the same REST API the frontend uses. Here the root endpoint, a filtered wardrobe query and the dashboard statistics are called against the containerised backend, which is reading from the containerised PostgreSQL — not from the in-memory store used by the tests:

fitpick — REST API running in Docker

```
$ curl -s localhost:8000/
{"message":"FitPick Backend Running","version":"1.0.0"}

$ curl -s "localhost:8000/clothing?category=Shoes" | python3 -m json.tool
[
    {
        "name": "Chelsea Boots",
        "category": "Shoes",
        "subcategory": "Boots",
        "colour": "Black",
        "season": "F/W",
        "brand": "Dr. Martens",
        "size": "9",
        "price": 160.0,
        "tags": [
            "Office",
            "Casual"
        ],
        "notes": "",
        "id": 12,
        "is_favourite": true,
        "times_worn": 10,
        "created_at": "2026-09-08"
    },

$ curl -s localhost:8000/stats/dashboard | python3 -m json.tool
{
    "total_items": 12,
    "total_outfits": 3,
    "favourites": 7,
    "most_worn": "Dark Wash Jeans",
    "most_worn_count": 25,
    "added_this_week": 3,
    "breakdown": [
        {
            "label": "Tops",
            "count": 4
        },
        {
            "label": "Bottoms",
    …
            "count": 2
        },
        {
            "label": "Shoes",
```

*Figure 12 — Smoke test of the running API: filtering and statistics served from PostgreSQL.*

## 7.4 Generated API documentation

FastAPI generates an OpenAPI description from the Pydantic schemas, so the API documents itself at `/docs`. This is the contract the React frontend is written against:

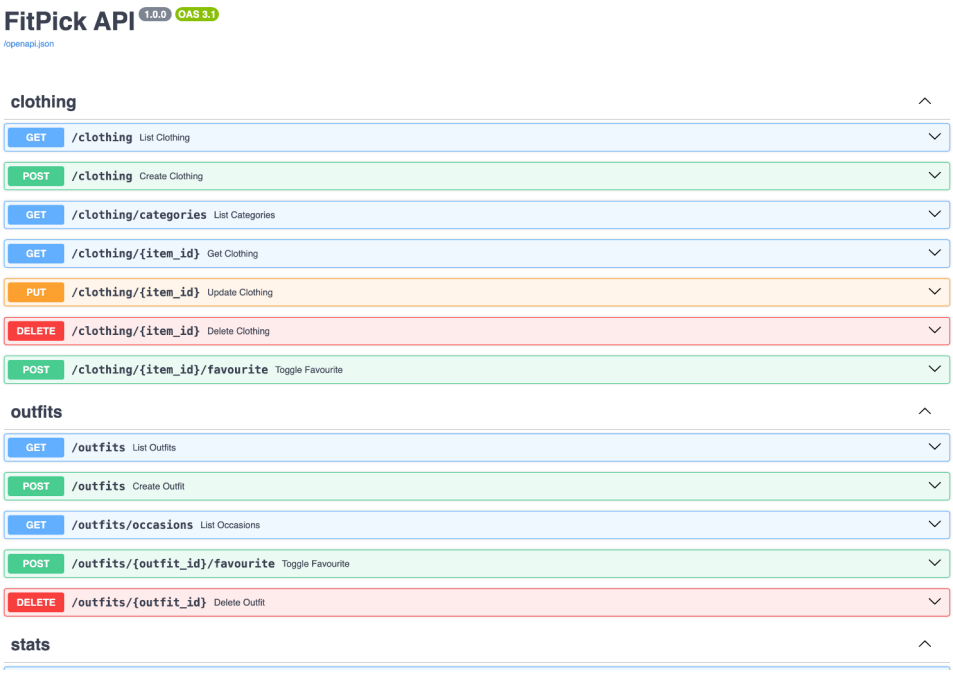



*Figure 13 — The 15 REST endpoints, generated from the code at http://localhost:8000/docs.*

## 7.5 Build instructions

*Everything, from a clean machine*

```
$ git clone https://github.com/nithi0307/fitpick.git
$ cd fitpick
$ docker compose up --build

#  frontend     http://localhost:5173
#  backend API  http://localhost:8000   (docs at /docs)
#  PostgreSQL   localhost:5433  db fitpick  user postgres
```

# 8. Screenshots of developed functionality

All six screens are implemented in React and were captured from the running application with the backend and PostgreSQL serving real data. The design system — royal blue #3B6FF6 on white cards over a light grey ground — is shared by every screen.

## 8.1 Login / Sign Up

A split screen: the form on the left, the product promise on the right. One form serves both sign-in and sign-up so the user never leaves the page.

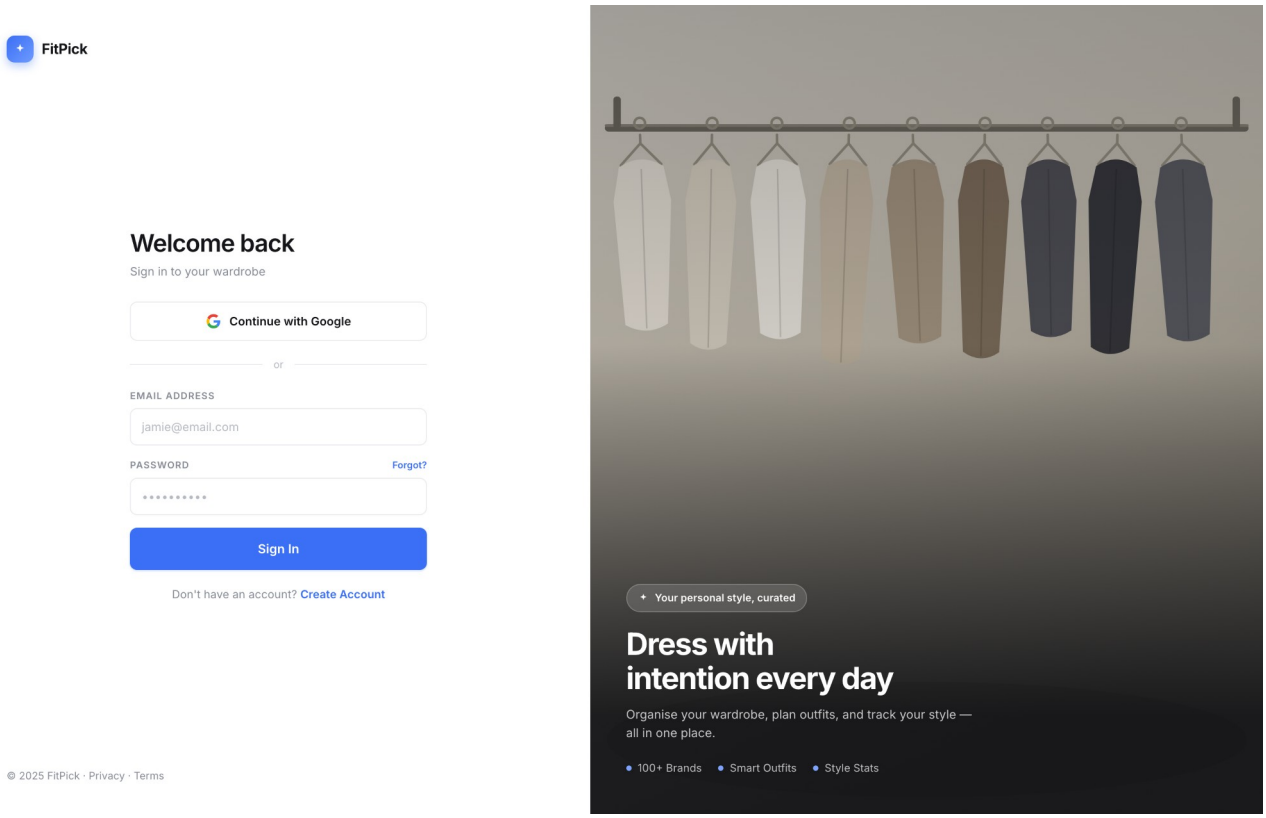



*Figure 14 — Login / Sign Up*

## 8.2 Dashboard

Four live figures across the top, recent additions and saved outfits on the left, quick actions and the wardrobe breakdown on the right. Every number is calculated by the backend from the real wardrobe.

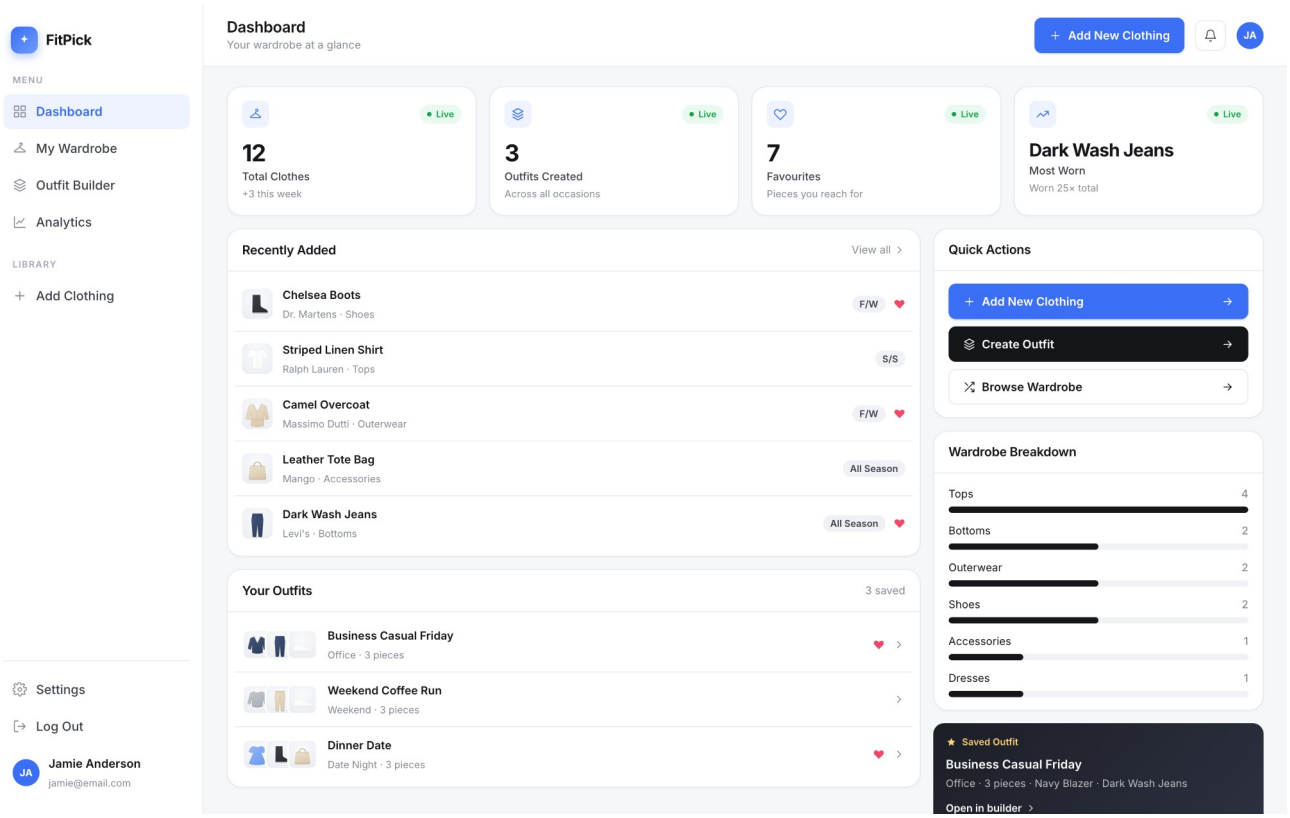



*Figure 15 — Dashboard*

## 8.3 My Wardrobe

The full collection as a card grid, with search, category and season filters and a sort order. Filtering is done by the API, so the same rules apply to every client.

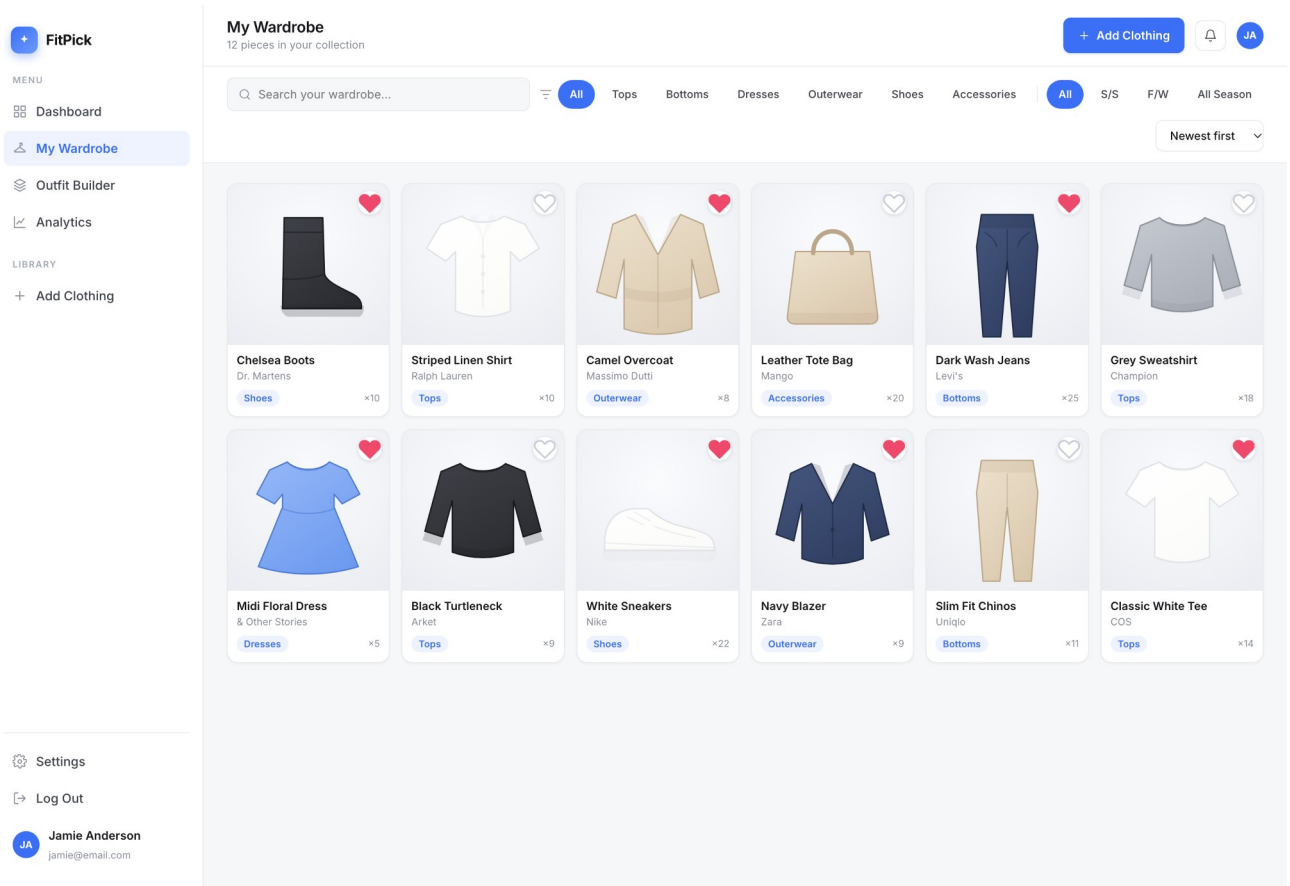



*Figure 16 — My Wardrobe*

## 8.4 Add Clothing Item

The capture form: name, category and subcategory, colour, season, brand, size, price paid, tags and notes, beside an optional photo upload.

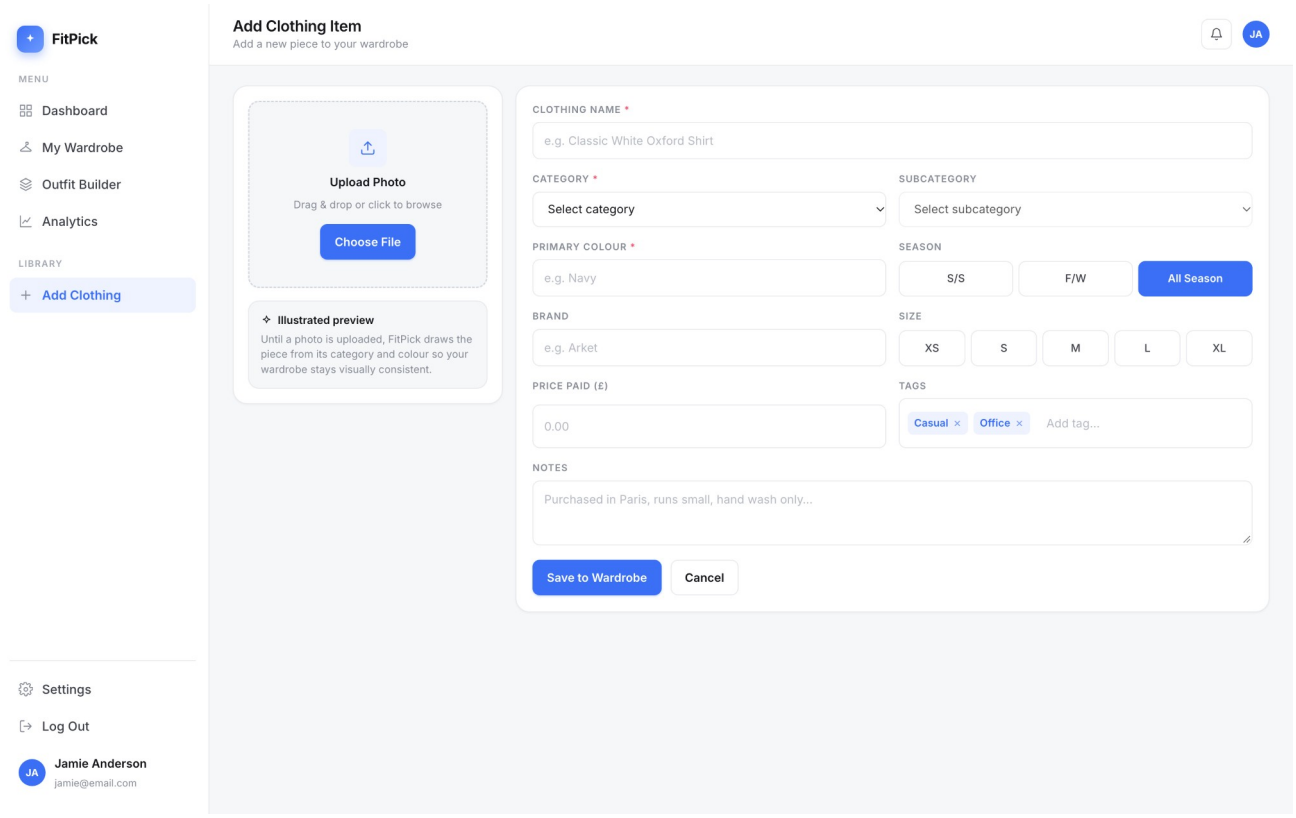



*Figure 17 — Add Clothing Item*

## 8.5 Outfit Builder

Three columns — pick pieces on the left, see the outfit take shape in the middle, name it and set its occasion on the right. Opening it with ?outfit=<id> loads a saved outfit for editing.

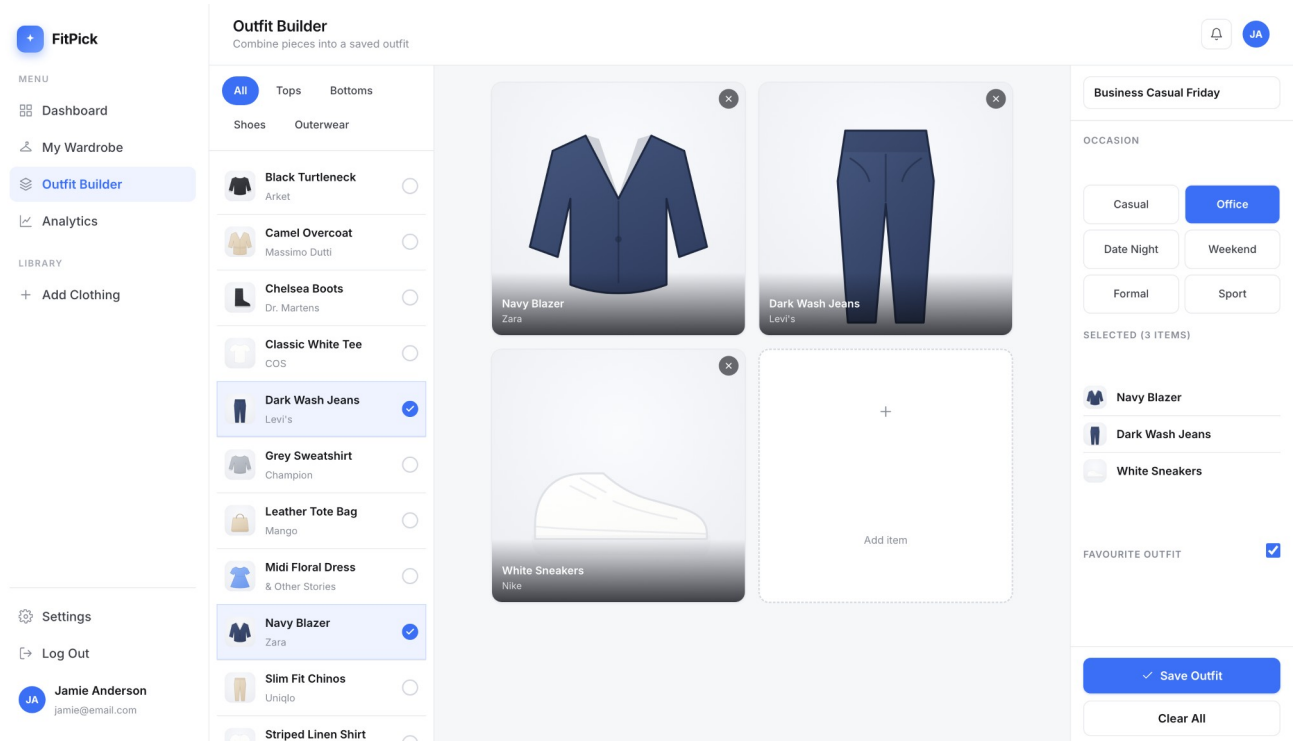



*Figure 18 — Outfit Builder*

## 8.6 Profile & Analytics

Account details beside wardrobe insight: total value, average wears per item, a category donut and a colour breakdown, all computed from the user's own items.

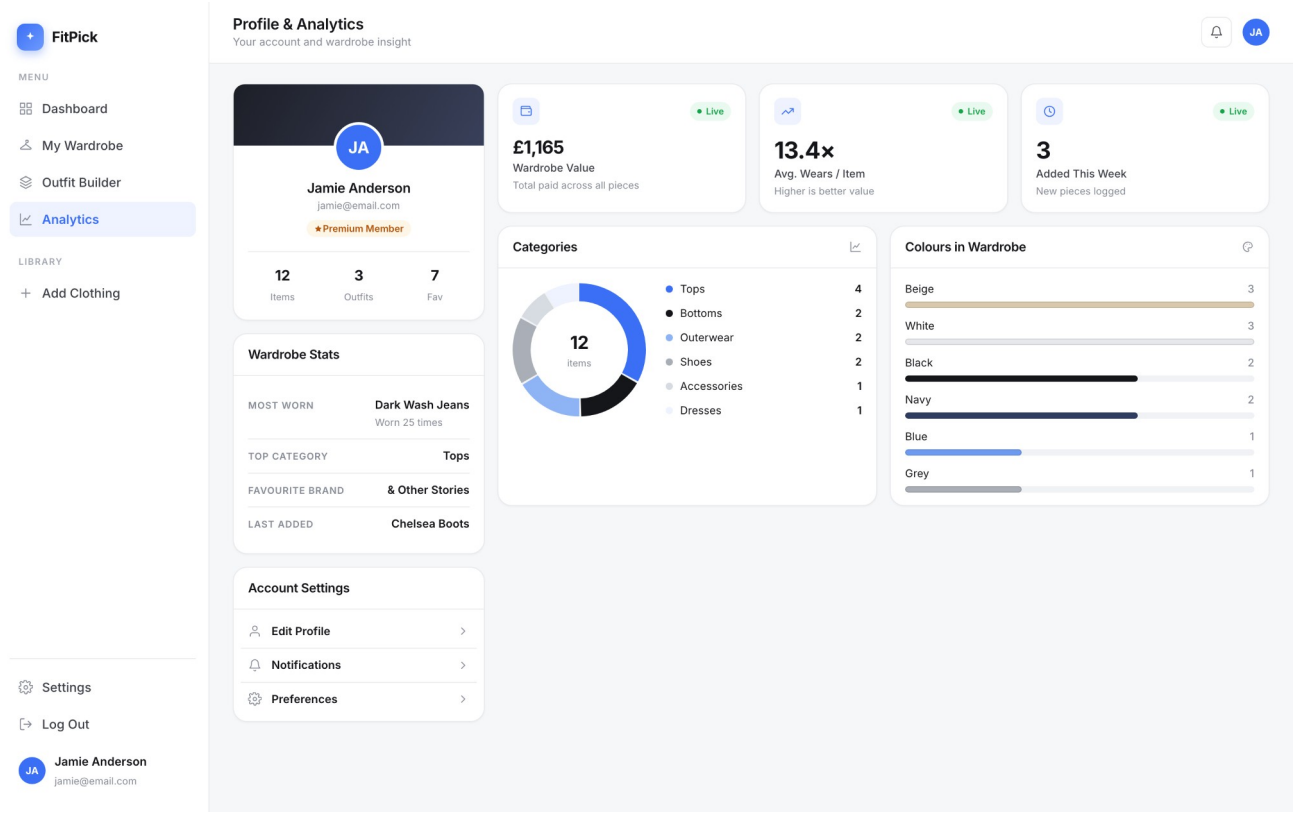



*Figure 19 — Profile & Analytics*

# 9. Tools and technologies used

## 9.1 Application stack

| Layer | Technology | Version | Why it was chosen |
|---|---|---|---|
| **Frontend** | React + Vite | 19.2 / 8.2 | Component model maps one-to-one onto the six designed screens; Vite gives instant rebuilds |
| **Routing** | React Router | 7.18 | Declarative route table; nested routes give every screen the sidebar automatically |
| **Backend** | FastAPI | 0.115 | Small typed REST framework with validation and OpenAPI documentation built in |
| **Validation** | Pydantic | 2.x | Models double as the API contract, so invalid data is rejected before it reaches storage |
| **Server** | Uvicorn | 0.34 | ASGI server used in development and inside the container |
| **Database** | PostgreSQL | 16 | The data is relational: users own items, outfits join items many-to-many |
| **DB driver** | psycopg | 3.2 | Modern PostgreSQL driver with native support for arrays and dict rows |
| **Containers** | Docker Compose | v2 | One command builds and starts all three services identically on any machine |

## 9.2 Quality and engineering tools

| Purpose | Tool | How it is used |
|---|---|---|
| **Test runner** | pytest 8.3 | Runs all 46 tests; configuration in pytest.ini |
| **HTTP test client** | Starlette TestClient / httpx | Sends real requests into the application for the integration tests |
| **Mutation testing** | mutmut 2.5 | Measures test quality against services/analytics_service.py |
| **Linting** | ESLint 10 | Checks the frontend on every change (npm run lint) |
| **Build** | Vite build | Produces the production bundle and catches import errors |
| **Version control** | Git + GitHub | Branching, history, issues and the MoSCoW project board |
| **Design** | Figma | The six screens were designed before being implemented |
| **Diagrams** | Draw.io (diagrams.net) | Architecture, DFD, ERD and UML diagrams; editable sources in docs/design/ |

## 9.3 Deliberate exclusions

A small stack was chosen on purpose. The following were considered and left out because they would add work and risk without serving any user story in this version:

- No AI or machine-learning outfit recommendation — outside the agreed scope.
- No JWT, OAuth or session store — the login screen is a front end only; authentication is listed as future work.
- No Nginx or reverse proxy — Vite and Uvicorn serve the two applications directly in development.
- No micro-services, message queue or cache — a three-tier layered design is the right size for this system.
- No state-management library (Redux, Zustand) — React's own state is enough for six screens.

Nothing in this report is claimed that is not in the repository. The code, tests, diagrams and screenshots are all at `github.com/nithi0307/fitpick`.